

# The tree: robotics system design at a glance

This page is the whole subject as one tree. Every branch is a family of ideas, every leaf is one idea, and the tag at the end of a line says where it is taught (L2 2.3 means level 2, section 2.3). Use it three ways:

1. **Before studying:** find the branch you are about to read, so you know where it sits.
2. **After studying:** cover the tree, redraw it from memory on paper, then compare. Do the whole tree once a week. The branches you cannot redraw are the ones to reread.
3. **When explaining:** walk the tree top down out loud. The structure carries you even when a word escapes you: "the brain has three questions: where am I, what next, how do I move".

Every robot is one tree. The root has three big branches, sense, think and act, plus the branches that describe the arrows between them and how the whole thing is run.

```

ROBOT SYSTEM
|
+-- SENSE (what comes in)                                L1 1.1, L2 2.1
|   |-- Looking outward (the world)
|   |   |-- camera: mono, stereo, depth (RGB-D)
|   |   |-- lidar: 2D scan, 3D point cloud
|   |   |-- radar, ultrasonic, microphone
|   |-- Looking inward (the robot itself)
|   |   |-- wheel encoders, joint encoders
|   |   |-- IMU: gyroscope + accelerometer
|   |   |-- motor current, temperature, battery gauge
|   |-- Absolute position aids
|   |   |-- GPS / GNSS, markers (ArUco), beacons (UWB)
|   |-- Safety sensors
|   |   |-- bump, cliff, emergency-stop button, light curtain
|   |-- The driver box: raw bytes -> timestamped message, in a named frame, at a rate L2 2.1
|
+-- THINK (from data to decision)                          L2 2.1
|   |-- Perception: "what is in the data?"
|   |   |-- detection, segmentation, clustering
|   |   |-- distance transform, tracking over time
|   |-- State estimation: "where am I, how fast?"          L2 2.1, tutorial 06
|   |   |-- odometry from wheels / IMU
|   |   |-- Kalman filter (predict, update, gain), EKF, particle filter
|   |   |-- localisation against a map, SLAM (map and pose together)
|   |-- Mapping: "what does the world look like?"
|   |   |-- occupancy grid, costmap layers (static, obstacle, inflation)
|   |   |-- semantic map, HD map
|   |-- Frames and time: "in whose coordinates, and when?" L2 2.3, 2.4
|   |   |-- map / odom / base_link / sensor frames, the TF tree
|   |   |-- timestamps, clock sources, simulated time
|   |-- Planning: "what should I do next?"                tutorial 03, 04
|   |   |-- global path: BFS, Dijkstra, A*, lattice, sampling (RRT)
|   |   |-- local path: obstacle avoidance, DWA / MPC / TEB
|   |   |-- task: which order, which bin, which station (task planner, assignment)
|   |   |-- multi-robot: reservations, priorities, lanes L4 4.4
|   |-- Control: "how do I move there?"                   tutorial 06
|   |   |-- PID, feedforward, MPC, trajectory tracking
|   |   |-- per-wheel / per-joint loops
|   |-- Supervisor: "what mode am I in, what if it fails?" L3 3.6
|   |   |-- state machine, behaviour tree
|   |   |-- mission logic, recovery behaviours, fault handling
|
+-- ACT (what goes out)                                    L1 1.1
|   |-- Actuators: wheel motors, steering, arm joints, gripper, elevator
|   |-- Motor drivers and bus: CAN, EtherCAT, serial; current / velocity / position loops
|   |-- Safety-rated stop: hardware e-stop, safety controller, watchdog L3 3.4, L4 4.8
|   |-- Outputs to people: lights, sounds, screen
|
+-- THE ARROWS (how boxes talk)                            L1 1.2, L2 2.2
|   |-- Interface = what travels + how often + which frame + which units

```

```

|   +-- Patterns
|   |   +-- topic: publish / subscribe (a noticeboard)
|   |   +-- service: request / reply (ask and wait)
|   |   +-- action: goal + feedback + result (order food, get updates)
|   +-- Middleware: ROS 2 / DDS, MQTT, gRPC / REST
|   +-- Delivery rules: reliable vs best-effort, queue depth, latest-value vs every-value  L3 3.2
|   +-- Three questions for any arrow: how fresh, how much, what if it stops          L1 1.4
|
+-- TIMING AND RELIABILITY (making it work every time)                                L3
|   +-- Latency budget: waiting + computing, added up, compared to physics          3.1
|   +-- Loops and rates: control 100-1000 Hz, perception 10-30 Hz, planning 1-10 Hz  3.2
|   +-- Concurrency: threads vs processes, races, locks, single-writer queues      3.3
|   +-- Real-time: worst case not average; no heap, locks or disk in the loop; watchdog 3.4
|   +-- Failure modes: sensor silent, sensor lying, network drop, crash, battery, slip 3.5
|   |   +-- for each: detect it, go to a safe state, degrade gracefully
|   +-- Testing pyramid: unit, simulation, replay of recordings, hardware-in-loop, field 3.7
|   +-- Observability: what to log, at what rate, replaying "why did it stop at 14:03" 3.8
|
+-- SCALE (from one robot to a fleet)                                                L4
|   +-- Fleet: shared map, traffic, task assignment, charging                      4.1, 4.4
|   +-- On-robot vs cloud: anything needed to stop safely never depends on the network 4.2
|   +-- Network: bandwidth, dropouts, send summaries not raw data, store-and-forward 4.3
|   +-- Maps and data: who owns the map, versions, what to collect, privacy          4.5
|   +-- Updates: over-the-air, staged rollout, rollback, config vs code             4.6
|   +-- Operations: dashboards, alerts, remote assistance / teleoperation          4.7
|   +-- Safety case: hazard table, safety-rated stop, ISO 13482 / 3691-4 / 26262     4.8
|   +-- Security: signed updates, authenticated commands                          4.9
|
+-- THE DESIGN METHOD (how to answer "design a robot that does X")                    L5 5.1
|   +-- 1. Requirements: must do, must never do (the five questions)
|   +-- 2. Numbers first: speed, space, sensor rates, battery, budget
|   +-- 3. Draw sense -> think -> act boxes
|   +-- 4. Label every arrow: what, how often
|   +-- 5. Go deep on the three hardest arrows: latency, failure, scale
|   +-- 6. Name the trade-offs: "X because Y matters more than Z"
|   +-- 7. Say what you would build first
|   +-- Worked examples: bin-picking arm, delivery robot, inspection drone,
|       200-robot warehouse, surgical arm, fixed-route shuttle                    L5 5.2-5.7

```

## The same tree in seven lines

When you have no paper, this is the version to hold in your head. Each line is one branch and its three or four children.

1. **Sense**: outward sensors, inward sensors, position aids, safety sensors.
2. **Think**: perceive, estimate state, map, frames and time, plan, control, supervise.
3. **Act**: actuators, drivers and bus, safety stop, outputs to people.
4. **Arrows**: interface, topic / service / action, middleware, delivery rules.
5. **Timing**: latency budget, rates, concurrency, real-time, failures, testing, logging.
6. **Scale**: fleet, robot vs cloud, network, data, updates, operations, safety case, security.
7. **Method**: requirements, numbers, boxes, arrows, deep dive, trade-offs, build first.

## The Think branch, one level deeper

Think is the branch you will be asked about most, so here it is with the one algorithm to name for each box:

```

THINK
+-- Perception ..... cluster lidar points into obstacles; detect boxes by colour
+-- State estimation ..... Kalman filter: predict with motion, update with sensor, gain decides trust
+-- Mapping ..... occupancy grid; inflation = distance transform (multi-source BFS)
+-- Frames and time ..... T_world_base * T_base_cam * p_cam; stamp everything

```

```
+-- Planning ..... A* on the costmap (Dijkstra + heuristic); local avoidance at 10-20 Hz
+-- Control ..... PID on velocity error, with anti-windup, at 100 Hz or faster
+-- Supervisor ..... state machine: idle, navigating, waiting, delivering, returning, fault
```

## Drill

---

Once a week: blank page, ten minutes, redraw the big tree from memory. Mark every branch you could not produce. Read those sections again. When you can redraw the whole tree, you can explain any part of a robot from the top down without notes.